

# Reporte Completo AhorraGo

Este documento consolida los cambios realizados en las fases 1, 2 y 3, m?s la auditor?a de calidad de datos.

# Fase 1 - Seguridad y Correcciones Iniciales

Fecha: 2026-05-31

## Objetivo

Corregir problemas críticos y de alto impacto inicial sin romper el MVP actual, evitando refactor masivo, cambios de arquitectura o migraciones de base de datos.

## Cambios Aplicados

### 1. Seguridad: API Key de Jumbo

- Se eliminó la API key hardcodeada de los scrapers de Jumbo.
- Los scrapers ahora leen JUMBO\_API\_KEY desde variables de entorno.
- Si falta la variable, el scraper falla con un mensaje claro sin exponer secretos.
- Se creó `.env.example` con la variable requerida.
- Se actualizó `.gitignore` para excluir `.env` y `.env.*`, manteniendo `.env.example` versionable.

Archivos:

- `app/scrapper_jumbo_real.py`
- `app/scrapper_jumbo_api.py`
- `.env.example`
- `.gitignore`

### 2. Búsquedas con LIMIT y OFFSET

- Se agregaron parámetros `limit` y `offset` a endpoints de búsqueda.
- Default: `limit=50`.
- Máximo permitido: `limit=100`.
- Se evitó carga ilimitada en las búsquedas principales de productos.
- El frontend ahora consulta con `limit=50&offset=0`.

Archivos:

- `app/main.py`
- `app/services.py`
- `frontend/index.html`

### 3. URLs Correctamente Escapadas

- Se creó helper centralizado para generar URLs de búsqueda con `urllib.parse.urlencode`.
- Se reemplazaron URLs generadas con `replace(" ", "%20")`.
- Las URLs ahora soportan tildes, ñ y caracteres especiales.

Archivos:

- `app/url_utils.py`
- `app/convertir_lider.py`
- `app/convertir_jumbo.py`
- `app/convertir_unimarc.py`
- `app/generar_catalogo.py`
- `app/services.py`

## 4. Índices Básicos en SQLite

- Se reforzó el script idempotente de índices.
- Se agregaron índices seguros para:
  - `productos.nombre`
  - `productos.producto_base`
  - `precios.producto_id`
  - `precios.supermercado_id`
- El script usa `CREATE INDEX IF NOT EXISTS`, por lo que no borra ni reconstruye datos.

Archivo:

- `app/scripts/agregar_indices.py`

## 5. Lock Real para Actualización de Productos

- Se reemplazó el lock manual por `filelock`.
- Se evita que dos ejecuciones del pipeline corran al mismo tiempo.
- Si ya existe una actualización en curso, el proceso sale con mensaje claro.

Archivos:

- `app/actualizar_productos.py`
- `requirements.txt`

## 6. Tests Iniciales

Se creó el primer set mínimo de tests para:

- Normalización de texto.
- Equivalencia entre `Leche Soprole 1L` y `Leche Soprole 1000ml`.
- URLs con tildes y ñ.
- Límites de búsqueda.
- Cálculo básico de ahorro.

Archivo:

- `tests/test_phase1.py`

## 7. Documento para Agentes

Se creó AGENTS.md con:

- Stack detectado.
- Reglas de trabajo para Codex.
- Comandos de ejecución.
- Comandos de testing.
- Reglas de seguridad.
- Definición de terminado.

Archivo:

- AGENTS.md

## Dependencias Agregadas

En requirements.txt:

```
filelock>=3.13.0  
pytest>=8.0.0
```

## Verificación Ejecutada

Comandos ejecutados:

```
venv\Scripts\python.exe -m pytest tests/test_phase1.py -q  
venv\Scripts\python.exe -m compileall app tests  
venv\Scripts\python.exe -m app.scripts.agregar_indices  
venv\Scripts\python.exe -c "from app.main import app; print(app.title)"
```

Resultados:

- Tests: 5 passed.
- Compilación Python: OK.
- Import de FastAPI app: OK.
- Índices SQLite aplicados correctamente.
- Verificación de JUMBO\_API\_KEY faltante: muestra error claro.

## Archivos Modificados o Creados

Modificados:

- .gitignore
- app/actualizar\_productos.py
- app/convertir\_jumbo.py
- app/convertir\_lider.py
- app/convertir\_unimarc.py
- app/generar\_catalogo.py

- `app/importar_csv.py`
- `app/main.py`
- `app/scraper_jumbo_api.py`
- `app/scraper_jumbo_real.py`
- `app/scripts/agregar_indices.py`
- `app/services.py`
- `frontend/index.html`
- `requirements.txt`

Creados:

- `.env.example`
- `AGENTS.md`
- `app/url_utils.py`
- `tests/test_phase1.py`
- `CAMBIOS_FASE_1.md`

## Riesgos Pendientes

- Aún existen `.all()` en endpoints de diagnóstico, estado de datos y lógica interna no crítica. No se tocaron para evitar ampliar demasiado esta fase.
- El pipeline completo de scrapers no fue ejecutado para evitar scraping externo y cambios masivos de datos.
- Hay un warning de Pydantic por uso de `@validator`; conviene migrarlo a `@field_validator` en una fase posterior.
- SQLite se mantiene como base actual, según la restricción de no migrar todavía.

## Siguiente Fase Recomendada

- 1 Optimizar endpoints pesados como `/diagnostico/calidad` y `/estado-datos`.
- 1 Agregar tests de integración con SQLite temporal.
- 1 Migrar validadores Pydantic a estilo v2.
- 1 Revisar límites y paginación en comparador y diagnósticos.
- 1 Revisar manejo de configuración con carga explícita de `.env` si el flujo local lo necesita.

# Fase 2 - Deuda Técnica y Tests de Integración

Fecha: 2026-05-31

## Objetivo

Reducir deuda técnica sin romper el MVP, enfocándose en `main.py`, normalización duplicada, matching de productos, endpoints pesados y tests de integración.

## Cambios Aplicados

### 1. Normalización compartida

- Se creó `app/normalizacion.py`.
- Se movieron funciones de normalización y comparación de claves desde `main.py`.
- `importar_csv.py` ahora usa `generar_producto_base` desde el módulo compartido.
- Se mantuvo compatibilidad con imports existentes usados por tests.
- Se normalizaron equivalencias de formato:

- 1L y 1000ml.

- 500g y 0.5kg.

- pack 6 y 6 unidades.

Archivos:

- `app/normalizacion.py`
- `app/main.py`
- `app/importar_csv.py`

### 2. Matching de productos

- Se creó `app/matching.py`.
- Se movió `candidato_compatible` y sus reglas relacionadas a un módulo dedicado.
- `main.py` ahora importa esa lógica en vez de contenerla directamente.
- Se agregaron tests unitarios de equivalencias.

Archivos:

- `app/matching.py`
- `app/main.py`
- `tests/test_matching.py`

### 3. Reducción segura de `main.py`

- Se retiró lógica de normalización y matching desde `main.py`.

- Los endpoints actuales se mantienen con las mismas rutas y estructura de respuesta.
- No se reestructuró la API ni se cambió la UX.

Archivo:

- `app/main.py`

## 4. Optimización de endpoints pesados

- `/diagnostico/calidad` ahora consulta columnas necesarias y usa `yield_per(500)` en vez de cargar objetos completos con `.all()`.
- `/estado-datos` ahora calcula conteos por supermercado con `GROUP BY` y conteos directos.
- Se mantuvo la forma de respuesta esperada por el frontend.

Archivo:

- `app/main.py`

## 5. Tests de integración mínimos

- Se agregó SQLite temporal en memoria para pruebas de API.
- Se prueba búsqueda básica con datos mínimos.
- Se prueba límite de resultados.
- Se prueba estado de datos con conteos básicos.

Archivo:

- `tests/test_integration.py`

## 6. Pydantic v2

- Se migró `@validator` a `@field_validator`.
- El warning de Pydantic v2 desaparece en la suite de tests.

Archivo:

- `app/schemas.py`

## 7. Dependencias

Se agregó `httpx`, requerido por `fastapi.testclient.TestClient` para los tests de integración.

Archivo:

- `requirements.txt`

## Comandos Ejecutados

```
$env:PATH = "$PWD\venv\Scripts;$env:PATH"; python -m pytest -q
$env:PATH = "$PWD\venv\Scripts;$env:PATH"; python -m compileall app tests
$env:PATH = "$PWD\venv\Scripts;$env:PATH"; python -c "from app.main import app; print(app.title)"
```

## Resultados

- Tests: 11 passed.
- Compilación: OK.
- Import de FastAPI app: OK, salida FastAPI.

## Archivos Modificados o Creados

Modificados:

- `app/importar_csv.py`
- `app/main.py`
- `app/schemas.py`
- `requirements.txt`

Creados:

- `app/normalizacion.py`
- `app/matching.py`
- `tests/test_matching.py`
- `tests/test_integration.py`
- `CAMBIOS_FASE_2.md`

## Riesgos Pendientes

- Todavía quedan `.all()` en rutas livianas (categorias, subcategorias) y en lógica interna acotada; no se tocaron para mantener esta fase pequeña.
- `main.py` aún contiene lógica de búsqueda y resumen de compra que podría separarse en una fase posterior.
- El matching sigue siendo heurístico; requiere más fixtures reales para cubrir casos de marcas, sabores, formatos familiares y productos con nombres ambiguos.
- No se ejecutó pipeline completo de scrapers para evitar cambios masivos de datos.

## Siguiente Fase Recomendada

- 1 Extraer búsqueda de productos desde `main.py` a un service dedicado.
- 1 Agregar fixtures realistas para matching por supermercado.
- 1 Cubrir `/productos/resumen-compra` con tests de integración.
- 1 Revisar endpoints internos con `.all()` restantes y paginar si empiezan a crecer.
- 1 Evaluar carga explícita de `.env` local para mejorar experiencia del scraper sin exponer secretos.



# Fase 3 - Matching Avanzado

Fecha: 2026-05-31

## Objetivo

Aumentar la precisión del matching de productos y del comparador sin cambiar la arquitectura principal, sin migrar la base de datos y sin modificar el frontend.

## Cambios Aplicados

### 1. Normalización avanzada

Se amplió `app/normalizacion.py` para reconocer equivalencias de formato:

- Volumen:
  - 1L = 1000ml
  - 1.5L = 1500ml
  - 2L = 2000ml
- Peso:
  - 1kg = 1000g
  - 0.5kg = 500g
  - 250g = 0.25kg
- Cantidad:
  - pack 6
  - 6 un
  - 6 unidades
  - x6

Esto reduce falsos negativos cuando distintos supermercados publican el mismo formato con unidades diferentes.

### 2. Extractor de atributos estructurados

Se agregó `extraer_atributos(nombre_producto)`, que devuelve señales estructuradas para el matching:

```
{
  "marca": "soprole",
  "volumen": 1000,
  "peso": None,
  "unidad": "ml",
  "cantidad": None,
  "categoria": "leche",
  "sabores": [],
  "tokens": [...]
}
```

Ejemplo cubierto por tests:

```
extraer_atributos("Leche Soprole Entera 1L")
```

Devuelve marca soprole, volumen 1000, unidad ml y categoría leche.

### 3. Matching Score 0-100

Se creó `matching_score(producto, candidato)` en `app/matching.py`.

El score combina:

- Marca.
- Volumen o peso normalizado.
- Cantidad o pack.
- Categoría/familia.
- `producto_base`.
- Intersección de palabras clave.
- Similitud textual con RapidFuzz.

`candidato_compatible` ahora mantiene filtros duros para evitar falsos positivos y usa el score como señal final.

### 4. RapidFuzz

Se agregó `rapidfuzz` a `requirements.txt` y se usa `fuzz.token_set_ratio` para evitar depender de comparaciones exactas de texto.

Esto mejora casos como:

- Coca Cola vs Coca-Cola.
- Reordenamientos de palabras.
- Diferencias menores en nombres publicados por supermercado.

### 5. Casos problemáticos cubiertos

Se agregaron tests para:

- Leche Soprole 1L vs Leche Soprole 1000ml.
- Coca Cola 1.5L vs Coca Cola 1500ml.
- Arroz Tucapel 1kg vs Arroz Tucapel 1000g.
- Pack 6 Coca Cola vs Coca Cola x6.
- Yogurt de sabores distintos, que no debe coincidir.

### 6. Fixtures reales

Se creó `tests/fixtures/productos_reales.json` con ejemplos representativos de Jumbo, Lider y Unimarc.

Incluye pares equivalentes y un caso negativo de yogurt con sabores distintos.

## Archivos Modificados o Creados

Modificados:

- app/normalizacion.py
- app/matching.py
- requirements.txt

Creados:

- tests/test\_matching\_score.py
- tests/test\_fixtures\_matching.py
- tests/fixtures/productos\_reales.json
- CAMBIOS\_FASE\_3.md

## Tests Agregados

- Tests unitarios para extractor de atributos.
- Tests unitarios para score de matching.
- Tests con fixtures reales.
- Tests negativos para evitar coincidencias incorrectas por sabor.

## Comandos Ejecutados

```
$env:PATH = "$PWD\venv\Scripts;$env:PATH"; python -m pytest -q
$env:PATH = "$PWD\venv\Scripts;$env:PATH"; python -m compileall app tests
$env:PATH = "$PWD\venv\Scripts;$env:PATH"; python -c "from app.main import app; print(app.title)"
```

## Resultados

- Tests: 19 passed.
- Compilación: OK.
- Import de FastAPI app: OK, salida FastAPI.

## Precisión Mejorada

La precisión mejora porque el matching ya no depende solo de texto o formato literal:

- Las unidades se comparan en una representación común.
- Los packs se detectan como cantidad.
- La marca se normaliza antes de comparar.
- RapidFuzz tolera diferencias de orden, guiones y texto adicional.
- Los sabores conocidos actúan como freno para evitar falsos positivos.

## Riesgos Pendientes

- El sistema de sabores aún es una lista curada y puede necesitar ampliación con datos reales.
- El umbral de `matching_score`  $\geq 68$  está calibrado con tests iniciales, pero debe validarse con más productos reales.
- Algunas categorías complejas, como detergentes, papel higiénico y promociones multipack, podrían requerir reglas específicas.
- No se ejecutó el pipeline completo de scrapers para evitar cambios masivos de datos.

## Próximos Pasos Recomendados

- 1 Medir precisión con una muestra real etiquetada manualmente.
- 1 Agregar fixtures para detergentes, papel higiénico, aceites y cafés.
- 1 Registrar `matching_score` en diagnósticos internos para revisar falsos positivos.
- 1 Ajustar umbrales por categoría si aparecen patrones distintos.
- 1 Crear un reporte de productos agrupados por `producto_base` para detectar errores de equivalencia.

# Auditoría de Datos

## Resumen Ejecutivo

- Calidad estimada de datos: Baja.
- Productos totales: 31124.
- Precios totales: 31139.
- Productos sospechosos detectados: 3.
- Producto\_base conflictivos: 1601.

## Conteos Base

- productos\_totales: 31124
- precios\_totales: 31139
- supermercados: 3
- categorias: 13
- subcategorias: 54
- productos\_sin\_precio: 0
- productos\_duplicados: 244
- productos\_sospechosos: 3
- producto\_base\_conflictivos: 1601

## Métricas de Matching

- porcentaje\_productos\_agrupados: 100.0
- porcentaje\_sin\_equivalencia: 79.94
- cantidad\_promedio\_equivalencias\_por\_producto\_base: 1.14
- supermercados\_cubiertos: 3
- supermercados\_con\_precio: Jumbo, Líder, Unimarc
- grupos\_producto\_base: 27418
- grupos\_con\_equivalencia: 2538

## Riesgos Encontrados

- precio extremadamente alto: 3

## Archivos Generados

- reports/auditoria\_datos.md
- reports/productos\_duplicados.csv

- `reports/productos_sospechosos.csv`
- `reports/producto_base_conflictivos.csv`

## Recomendaciones

- Revisar primero `producto_base_conflictivos.csv`, porque afecta directamente la precision del comparador.
- Corregir productos sin precio o sin supermercado antes de agregar funcionalidades de usuarios.
- Validar manualmente grupos grandes de `producto_base` antes de usarlos para recomendaciones.
- Mantener esta auditoria como paso obligatorio antes de cambios grandes en datos.

## Nota

Esta auditoria no modifica datos. Solo lee la base y genera reportes.